

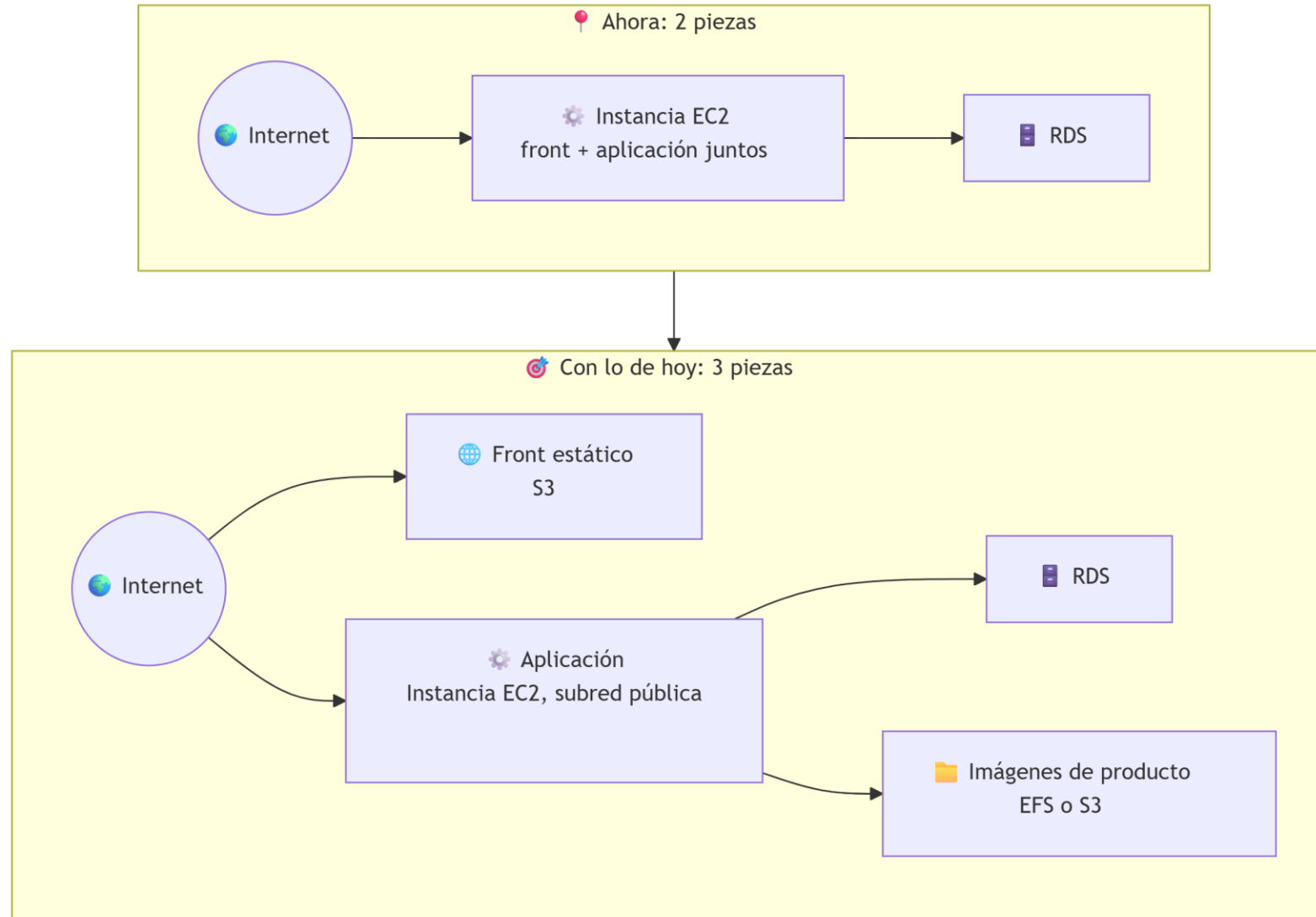
Primera arquitectura cloud completa

Todas las piezas sueltas, juntas por fin

Tienes ya todas las piezas sueltas: red con capas públicas y privadas, instancias, almacenamiento de objetos y compartido, y una base de datos gestionada sin credenciales en el código. Hoy no aprendes ningún servicio nuevo — hoy las juntas todas en una sola arquitectura.

Es el cierre natural de todo lo construido desde la primera sesión, y el punto de partida de todo lo que viene: alta disponibilidad, monitorización, coste, automatización.

De dos piezas a tres: separas también el front



Cada pieza solo habla con la siguiente

El front y la aplicación reciben tráfico de internet por caminos distintos —el front directamente desde S3, la aplicación desde su propia instancia—, y solo la aplicación tiene permiso para hablar con la base de datos.

Es el mismo principio de mínimo privilegio que has aplicado con los grupos de seguridad del Tema 2, ahora aplicado a la arquitectura entera.

CORS: por qué el backend necesita permitirlo

El front vive en una URL de S3 y la aplicación en la IP de una instancia EC2. Para el navegador son orígenes distintos: un origen se define por protocolo, dominio y puerto — con que uno solo cambie, ya cuentan como orígenes diferentes.

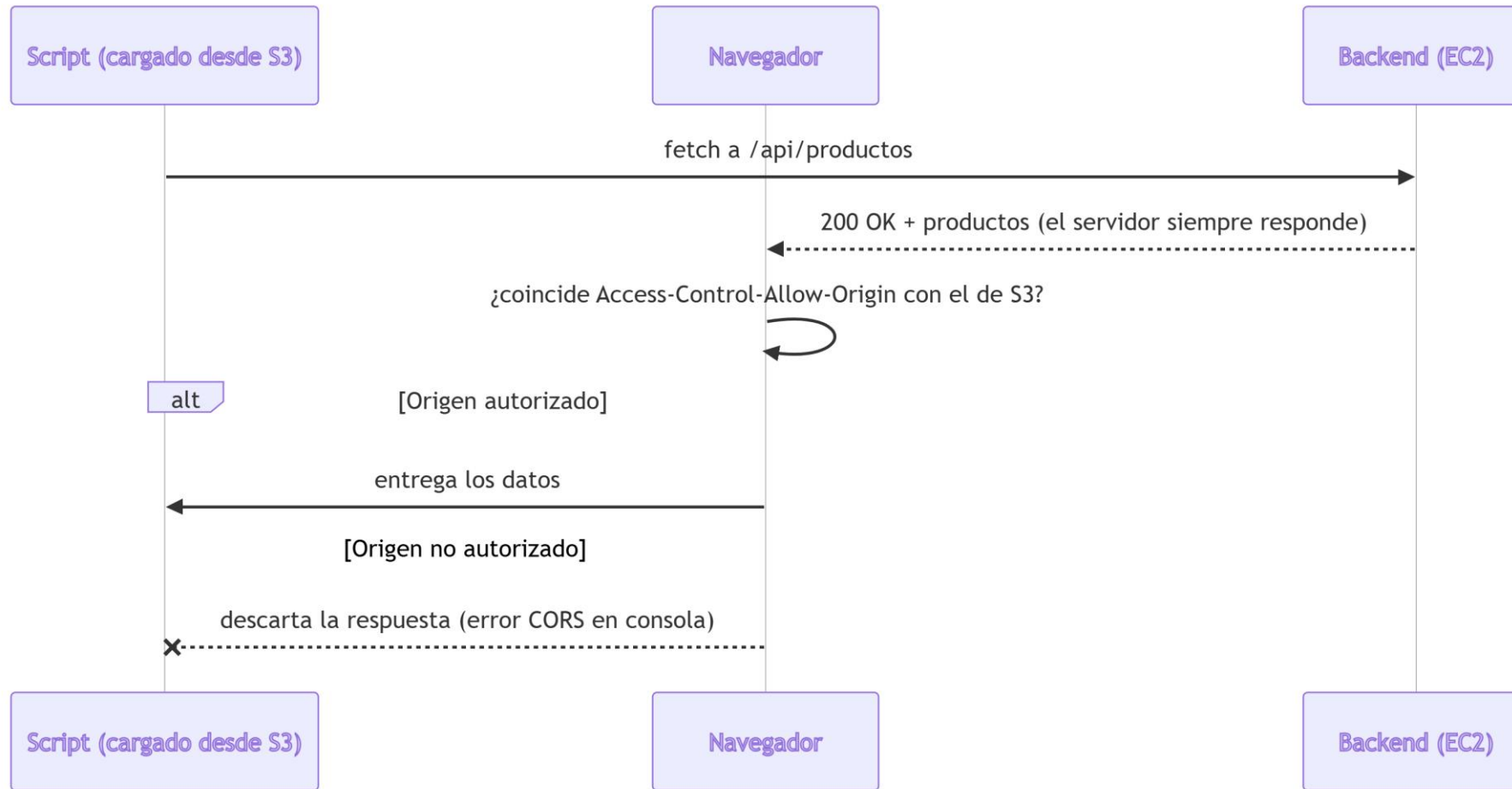
	Protocolo	Dominio	Puerto
Front (S3)	http	escaparate-front-xxx.s3-website-...	80
Backend (EC2)	http	3.84.12.9	8080

Por defecto, un script cargado desde un origen no puede leer la respuesta de una petición hecha a otro origen. Esta restricción no protege al servidor: te protege a ti.

El banco y la pestaña maliciosa

- Cualquier script puede pedir datos a cualquier URL, aunque no sea la suya.
- El navegador guarda las cookies por dominio: si tienes sesión en el banco, cualquier petición a ese dominio lleva tu cookie puesta.
- La petición del script malicioso llega al banco con tu cookie, y el banco responde como si fueras tú.
- Aquí actúa la restricción: el navegador deja salir la petición, pero no deja que el script la lea.

Cuándo se comprueba: después de responder



El servidor responde igual en los dos casos. Es el navegador quien, ya con la respuesta en la mano, decide si se la pasa al script o la descarta.

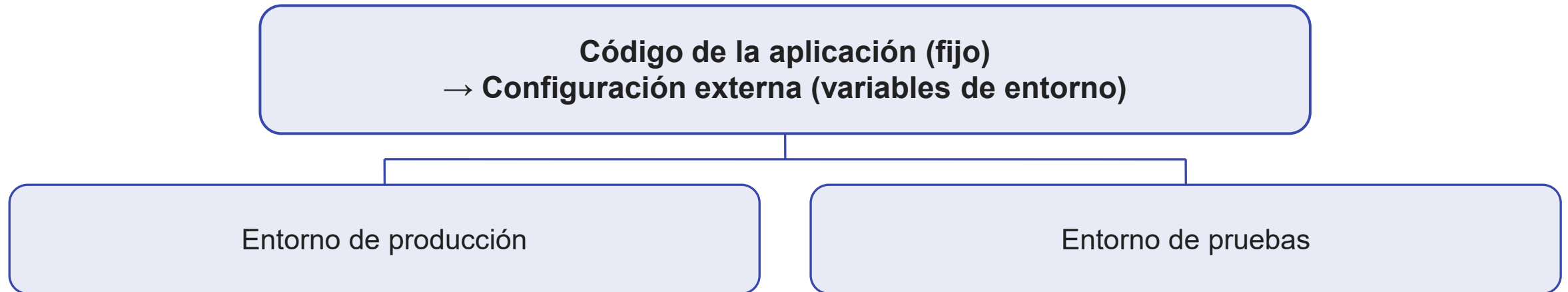
Separación de responsabilidades

Capa	Responsabilidad	Lo que NO hace
Front (S3)	Servir HTML, CSS y JavaScript	No ejecuta lógica de negocio, no toca la base de datos
Aplicación (EC2)	Procesar peticiones, aplicar reglas de negocio	No sirve el front estático, no almacena datos permanentes
Base de datos (RDS)	Guardar y devolver datos de forma consistente	No sabe presentar nada, no toma decisiones de negocio

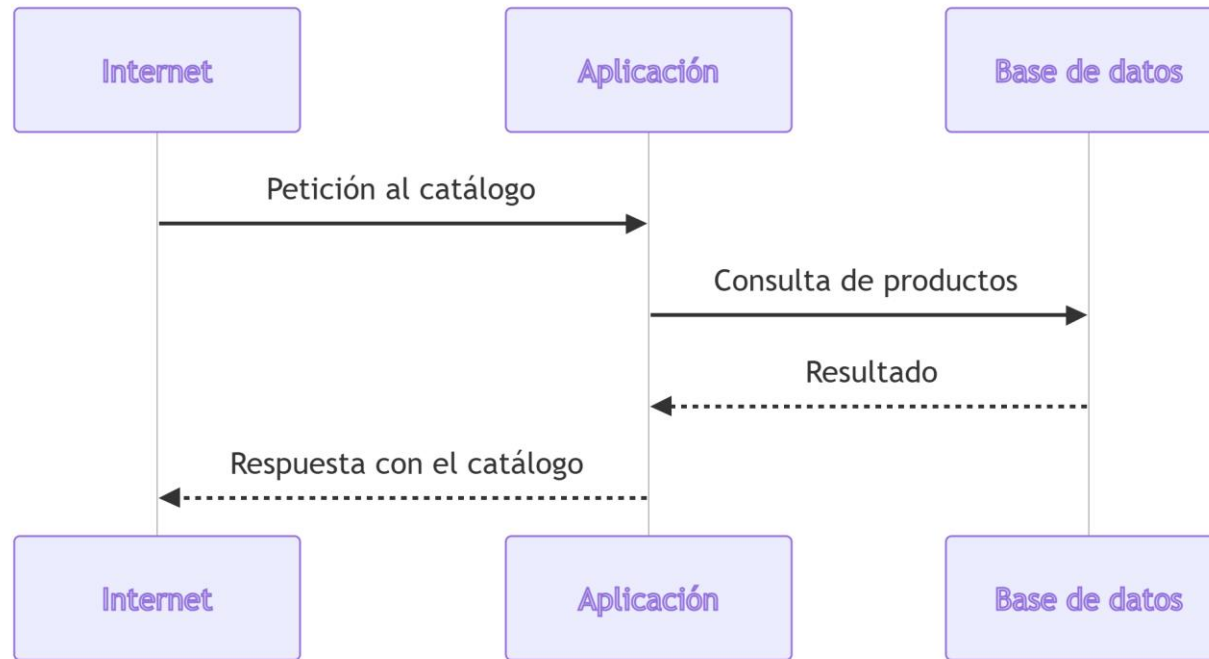
Si mañana necesitas escalar solo la aplicación, no tocas ni el front ni la base de datos — están desacoplados. Esa independencia hace posible el Tema 4.

Configuración externa

El endpoint de RDS, la ruta del sistema de ficheros, la URL del front: ninguno vive escrito dentro del código. Da igual que el dato sea público o un secreto — los dos llegan a la aplicación como variables de entorno en el arranque.



Cadena de dependencias entre capas



Si la base de datos no está lista cuando arranca la aplicación, la aplicación falla al conectarse — la solución casi nunca es cambiar el orden, sino reintentar la conexión unos segundos.

Qué se rompe cuando una pieza se mueve

Qué se mueve	Qué se rompe si no se actualiza la referencia
Endpoint de la base de datos	La aplicación no puede conectar
Ruta del sistema de ficheros compartido	La aplicación no encuentra las imágenes de producto
Dirección del front	Los enlaces del catálogo dejan de resolver

El fallo no está en la pieza que se movió — está en la referencia que se quedó apuntando al sitio viejo.

Puntos únicos de fallo (SPOF)

Un punto único de fallo es cualquier pieza tal que, si falla ella sola, se cae el sistema entero. Identificarlos es el objetivo de la Parte B de la actividad de hoy.

Pieza	Por qué es un SPOF hoy
La instancia EC2 de la aplicación	Es una sola — si se para, no hay copia que responda
La instancia RDS (sin Multi-AZ)	Un fallo de zona tumba la base de datos entera
La región donde despliegas todo	Un fallo a ese nivel se lleva las tres capas a la vez

Tener SPOF hoy no es un error de diseño

Es el punto de partida

Toda arquitectura empieza con puntos únicos de fallo; lo que la hace madura no es no tenerlos desde el primer día, sino saber nombrarlos y decidir cuáles merece la pena resolver primero. Hoy los identificas; en el Tema 4 empiezas a eliminarlos.

Ideas clave

- Arquitectura de tres capas: front (S3), aplicación (EC2) y datos (RDS) — cada capa solo habla con la siguiente, mínimo privilegio aplicado a toda la arquitectura.
- CORS es la excepción explícita que el backend concede a un origen concreto; el bloqueo lo aplica siempre el navegador, después de que el servidor ya haya respondido.
- La separación de responsabilidades permite escalar o cambiar una capa sin tocar las demás.
- La configuración externa (variables de entorno) permite que el mismo código sirva para varios entornos.
- Si una dependencia tarda, la solución habitual es reintentar la conexión, no rehacer el despliegue a ciegas.
- Un punto único de fallo (SPOF) es cualquier pieza cuya caída tumba el sistema entero. Una arquitectura resiliente sigue funcionando cuando uno de esos puntos falla.